

# Engineering Guide

Handover notes for the next developer — how the system fits together, what it assumes, and where it will bite.

You have inherited a force-measurement dashboard for the SMS Lab climbing wall. Four three-axis load-cell boards measure what a climber pulls on each hand and foot; a Python backend reads them over USB; a React frontend turns the readings into Newtons, charts them live, records sessions to disk, and computes jump height.

This guide is not an API reference — `ARCHITECTURE.md` in the repository root already lists every file and every exported function. What follows is the part that is not written down in the code: why the pieces are arranged the way they are, which assumptions are load-bearing, what will look like a bug but isn't, and what to check before you trust a number the dashboard prints.

## 1. What to read, and in what order

The repository documents itself in four Markdown files at the root. They are maintained and current; do not re-derive from the source what is already written there.

| File                                 | Covers                                             | Read it                       |
|--------------------------------------|----------------------------------------------------|-------------------------------|
| <code>README.md</code>               | Setup, running, troubleshooting                    | Day one                       |
| This guide                           | Orientation, invariants, traps, open questions     | Day one                       |
| <code>ARCHITECTURE.md</code>         | Every file, every exported function                | When hunting for something    |
| <code>measuring_pipeline.md</code>   | Raw line → Newtons → storage → jump height         | Before touching the data flow |
| <code>calibration_pipeline.md</code> | The wizard and its maths                           | Before touching calibration   |
| <code>CLAUDE.md</code>               | The same rules, compressed, addressed to AI agents | If you work with one          |

The four LaTeX guides in `docs/` (system quick start, user guide, calibration quick start, calibration guide) are written for the people who *use* the wall, not for you. Read the user guide once anyway: it is the fastest description of what the software is supposed to do.

**`docs/` is gitignored.** The repository's `.gitignore` ignores the whole folder, so every LaTeX source and every built PDF — including this guide — exists only on the machine that made it. If you want the next person after you to receive it, either un-ignore the sources (keeping the PDFs out, or in, as you prefer) or move this file into the tracked part of the tree. It is a one-line change and it is the difference between a handover document and a local file.

## 2. The system on one page

Three things run, and only three:

1. **The boards.** Four PhidgetBridge amplifiers, three channels each (X, Y, Z) — twelve channels total — plugged into a USB hub, plugged into the laptop. There is no Raspberry Pi, no microcontroller firmware, and no Web Serial. Older comments that mention any of those are stale; trust the code.

2. **The backend** (`backend/app.py`, Flask, port 5001). It opens the twelve channels, holds the latest value of each, and broadcasts them over a WebSocket every 10 ms. It also computes jump height, stores recordings, and keeps the calibration files.
3. **The frontend** (`climbing_wall_website/`, React + Vite, port 5173). It converts raw readings to Newtons, draws the charts, runs the calibration wizard, and owns the zero.

What travels over the WebSocket is deliberately dumb: one line per sample, twelve comma-separated *raw voltage ratios* of order  $10^{-6}$ , at roughly 100 Hz. No units, no calibration, no coordinate transform. Everything that turns those numbers into physics happens in the browser, in `src/utils/serialParser.ts`.

**A consequence worth internalising.** Recording CSVs contain *calibrated Newtons*, not raw ratios — the conversion happened in the browser before the data was ever sent to be saved. If the calibration was wrong when a session was captured, that session is permanently wrong; there is no raw archive to re-derive it from. Check the calibration before a measurement campaign, not after.

### 3. Getting it running

Two terminals, always.

```
» cd backend
» source venv/bin/activate
» python app.py
```

(Windows: `venv\Scripts\activate`)  
→ Running on `http://127.0.0.1:5001`

```
» cd climbing_wall_website
» npm run dev
```

→ `http://localhost:5173`

Vite proxies `/api` and the `/api/stream` WebSocket to port 5001, so the browser only ever talks to 5173. Two details in `vite.config.ts` are load-bearing and have both been debugged the hard way already: the stream rule must come *before* the generic `/api` rule (otherwise the WebSocket upgrade is caught by the plain HTTP rule), and the target is spelled `127.0.0.1`, not `localhost` (Node resolves `localhost` to `::1` first, Flask's dev server binds IPv4 only, and the proxy gets `ECONNREFUSED`).

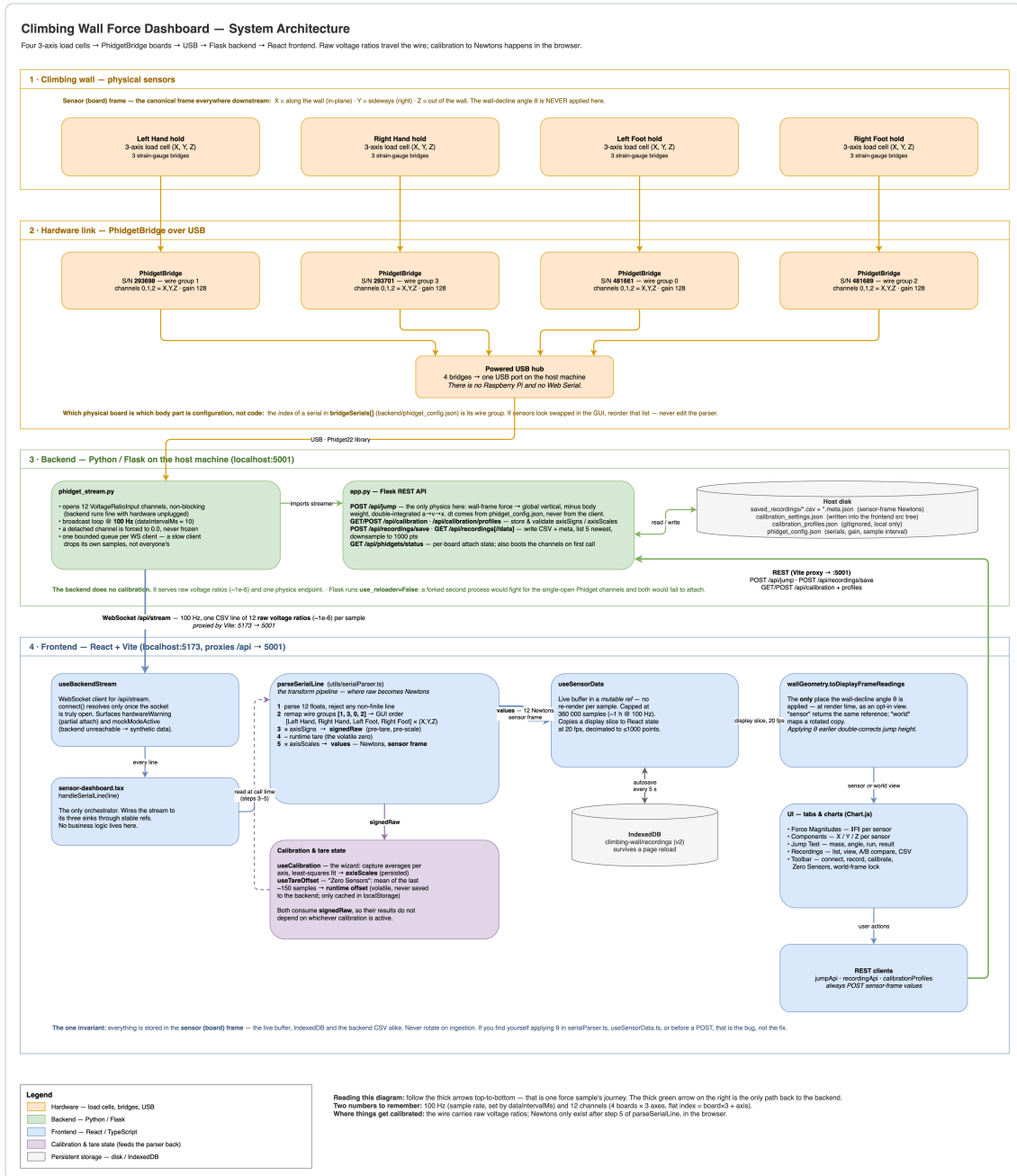
#### 3.1. Working without hardware

With no boards attached the app falls back to **demo mode**: press **Start Recording** and `useSensorData` generates synthetic values. This is enough to develop nearly everything — layout, charts, recordings, comparison, exports. It cannot do three things, and none of them is a bug:

- **Zero Sensors stays disabled.** Mock data bypasses `parseSerialLine` entirely, so the tare's sample ring is never fed.
- **The calibration wizard cannot capture.** Same reason; a capture times out after 8 s with no samples.
- **Jump height is fake.** Mock mode returns a random 20–120 cm rather than calling the backend.

#### 3.2. With hardware

- The Phidget driver must be approved once per machine (macOS: **System Settings** → **Privacy & Security**; Windows: install the Phidget22 drivers). Without it, no channel ever attaches.
- **Exactly one program may hold the boards.** They are single-open. A second `app.py`, or the Phidget Control Panel left running, and nothing attaches — while the backend log looks perfectly healthy.



**Figure 1.** The full data flow, board serial numbers through to charts. Generated from `docs/diagrams/system_architecture.drawio`; its per-board serial-to-hold mapping matches the live backend/`phidget_config.json`, *not* the `DEFAULT_CONFIG` fallback in `phidget_stream.py`, which lists the same serials in a different order.

- Restarting the backend requires re-clicking **Connect Device** in the browser. The socket is gone; the UI does not reconnect itself.
- Boards may be plugged in before or after the backend starts. Channels are opened non-blocking and attach whenever hardware appears, usually within a second or two.

## 4. The one invariant

**Everything is stored in the sensor (board) frame. Never rotate on ingestion.**

The wall is mounted with a backward decline of  $\theta$  degrees from vertical ( $16^\circ$  by default). The load cells are bolted to the wall, so their axes tilt with it:

| Sensor frame — canonical, what everything stores   |                                                             |
|----------------------------------------------------|-------------------------------------------------------------|
| $X$                                                | in-plane, along the wall, positive up-slope                 |
| $Y$                                                | sideways, positive right                                    |
| $Z$                                                | normal to the wall surface, positive out toward the climber |
| World frame — derived, display only                |                                                             |
| $X_{\text{world}} = X \cos \theta - Z \sin \theta$ | true vertical, up                                           |
| $Z_{\text{world}} = X \sin \theta + Z \cos \theta$ | true horizontal, out of the wall                            |
| $Y_{\text{world}} = Y$                             | unchanged                                                   |

The live buffer, IndexedDB, and the backend CSV all hold sensor-frame Newtons. The rotation is applied at *render time* only, by `toDisplayFrameReadings` in `wallGeometry.ts`, and only when the user has explicitly opted into the world view.

$\theta$  legitimately appears in exactly three places:

1. **In the calibration wizard**, to split one hang into its  $X$  and  $Z$  components ( $W \cos \theta$  and  $W \sin \theta$ ).
2. **In the world-frame display**, as the opt-in rotation above.
3. **In the backend jump algorithm**, which projects wall-frame forces to global vertical — see the open question in section 12.

Nowhere else. This invariant was inverted once already and produced double-corrected jump heights that looked plausible and were wrong. If you find yourself applying  $\theta$  in `serialParser.ts`, in `useSensorData.ts`, or anywhere before a `POST`, stop: that is the bug you are about to introduce, not the fix you are looking for.

## 5. Data contracts

### 5.1. Twelve channels, one order

Every length-12 array in the frontend is in **GUI-slot order**:

[Left Hand, Right Hand, Left Foot, Right Foot]  $\times$  (X, Y, Z), flat index = board \* 3 + axis

The hardware streams a *different* order (group 0 = Left Foot, 1 = Left Hand, 2 = Right Foot, 3 = Right Hand), and `SENSOR_GROUP_ORDER = [1, 3, 0, 2]` in `serialParser.ts` remaps it. Which physical board is which group is not in the code at all — it is `bridgeSerials` in `backend/phidget_config.json`, where the list index is the group number. **If sensors appear swapped in the GUI, reorder that list.** Nobody should ever need to edit `SENSOR_GROUP_ORDER`.

Recording CSVs follow the GUI order too: S1 is Left Hand, S2 Right Hand, S3 Left Foot, S4 Right Foot.

## 5.2. Two representations of a sample

| Name                   | What it is                                          | Magnitude      | Who consumes it                                               |
|------------------------|-----------------------------------------------------|----------------|---------------------------------------------------------------|
| <code>signedRaw</code> | Post axis-sign, <i>pre</i> -tare, <i>pre</i> -scale | $\sim 10^{-6}$ | The calibration wizard and the zero                           |
| <code>values</code>    | Fully calibrated<br>Newtons, sensor frame           | $\sim 10^2$    | Everything else: buffer, charts,<br>IndexedDB, CSV, jump test |

The split is the reason the wizard and the tare work at all: both average `signedRaw`, so their measurements do not depend on whatever offset or scale happens to be active while they run. Feeding them `values` would make each calibration depend on the previous one. The transform itself is a fixed four-step chain:



**Figure 2.** `parseSerialLine` in `serialParser.ts`. The tap after the sign step is what keeps calibration and tare independent of each other.

## 5.3. Sample numbers are not indices

Each buffered reading carries an absolute, monotonically increasing `sampleNumber`. The buffer is trimmed when it exceeds 360 000 samples, so array indices shift underneath you; sample numbers do not. The jump test marks its window by sample number for exactly this reason, and **Start Fresh** resets the numbering, which makes a stale jump window unreachable rather than silently wrong.

The backend’s recording window (`?from=` and `?to=` on the data endpoint) is the opposite: those are 0-based *row positions* in the file, not sample numbers. The docstring on `_slice_and_downsample` says so; believe it.

## 6. Where to change what

| You want to                                            | Touch                                                                                                                                                                    |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Fix a sensor showing up as the wrong body part         | <code>backend/phidget_config.json</code> → <code>bridgeSerials</code> . No code change.                                                                                  |
| Change the sample rate                                 | <code>phidget_config.json</code> → <code>dataIntervalMs</code> . This is also the single source of truth for the jump test’s <i>dt</i> .                                 |
| Change the amplifier gain                              | <code>phidget_config.json</code> → <code>bridgeGain</code> . Invalidates every saved scale — recalibrate afterwards.                                                     |
| Change how raw becomes Newtons                         | <code>src/utils/serialParser.ts</code> .                                                                                                                                 |
| Change the calibration maths                           | <code>src/utils/calibration.ts</code> ; the wizard’s UI is <code>dashboard/CalibrationModal.tsx</code> , its capture machinery is <code>hooks/useCalibration.ts</code> . |
| Change the zero                                        | <code>src/utils/runtimeOffset.ts</code> plus <code>hooks/useTareOffset.ts</code> .                                                                                       |
| Change anything about the wall angle or the world view | <code>src/utils/wallGeometry.ts</code> . Extend <code>wallDeclineSelfCheck()</code> in the same commit.                                                                  |
| Change buffering, the UI sync rate, or autosave        | <code>hooks/useSensorData.ts</code> .                                                                                                                                    |

| You want to                  | Touch                                                                                                                                                          |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Change a chart               | <code>utils/dataProcessing.ts</code> (data building),<br><code>utils/chartOptions.ts</code> (behaviour),<br><code>components/dashboard/tabs/*</code> (layout). |
| Add a backend endpoint       | <code>backend/app.py</code> , plus a thin client in <code>src/Utils/*Api.ts</code> .<br>Components never fetch directly.                                       |
| Change the jump physics      | <code>backend/app.py</code> → <code>calculate_jump_height_with_angle</code> .                                                                                  |
| Wire a new hook into the app | <code>components/sensor-dashboard.tsx</code> — and nothing else.                                                                                               |

`sensor-dashboard.tsx` is the only orchestrator in the frontend. It instantiates the hooks, routes each incoming line to its three sinks through stable refs (which is how the circular dependency between the stream and its consumers is broken), and composes the layout. No business logic belongs in it. When you are tempted to put "just one calculation" there, that calculation belongs in a hook or a util.

## 7. Calibration, profiles, and the zero

Three layers, deliberately separated, and the separation is the design:

| Layer              | What                                                                 | Where it lives                                                                                                                   |
|--------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Active calibration | <code>axisSigns</code> and <code>axisScales</code> , 12 numbers each | <code>src/config/calibration_settings.json</code> (written by the backend), mirrored in <code>localStorage["calibration"]</code> |
| Profile library    | Named calibrations you can load                                      | <code>backend/calibration_profiles.json</code> — gitignored, so a fresh clone has none                                           |
| Zero (tare)        | A volatile offset in signed-raw space                                | <code>localStorage["tareOffset"]</code> only.<br>Never saved with the calibration, never sent to the backend                     |

The zero was part of the saved calibration once and was deliberately pulled out. What stays in the calibration is only what genuinely belongs to the hardware: the sign of each axis and its Newtons-per-raw-unit slope. Where zero happens to sit today is a property of the moment — temperature, a hold that has been leaned on, a cable being dressed — so it is measured live from **Zero Sensors** (about 150 samples, roughly 1.5 s) and cached only so a page reload does not lose it.

Note the ordering consequence: the tare is applied *before* the scale, in signed-raw space, which is why it stays valid when you change the scale and why the wizard is unaffected by whatever zero is in effect.

**$\theta$  is not part of the calibration.** It is stored separately (`localStorage["cw:wallDeclineDeg"]`) and is only needed *during* a hang capture, to interpret how much of the hung weight landed on *X* versus *Z*. The fitted scales are an intrinsic sensor property; in the ideal orthogonal model the  $\cos \theta$  and  $\sin \theta$  factors cancel in the least-squares slope. Changing the wall angle does not require recalibrating.

Editing a profile's numbers by hand is intentionally not offered. The only way to change a profile's values is to load it, recalibrate against real weights, and save the changes back — the **Recalibrate** button in the profiles modal does exactly that. If a future request asks for raw number entry, the answer that was already given once is no.

## 8. Decisions that look like bugs

Every item here was a deliberate choice. Each has a failure mode you will reintroduce if you "clean it up".

| The odd thing                                            | Why, and what breaks if you undo it                                                                                                                                                                                                                               |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The live buffer is a mutable ref, pushed to in place     | At 100 Hz, React state would mean thousands of re-renders per second. A 50 ms interval copies a display slice instead (~20 fps). Never hand <code>allSensorDataRef.current</code> to React state or Chart.js directly.                                            |
| <code>decimate</code> sometimes returns its input array  | Same reference means memoised chart components can skip work. But with the window set to "all", that input <i>is</i> the live buffer, so <code>buildDisplaySlice</code> copies in that case — otherwise Chart.js would read an array being mutated underneath it. |
| Charts are capped at 1000 points, twice                  | Once in the frontend ( <code>decimate</code> ), once in the backend ( <code>MAX_CHART_POINTS</code> ). Even-stride selection plus the 5-sample moving average is visually faithful and keeps payloads and render cost bounded.                                    |
| The buffer is trimmed in 36 000-sample chunks            | Capped at 360 000 (~1 hour). Without it, an all-day session grows unbounded and the 5 s IndexedDB autosave re-serialises the lot each time.                                                                                                                       |
| A slow WebSocket client drops its own samples            | Each client has a bounded queue; a full queue drops rather than blocks, so one stalled browser tab cannot stall the broadcast thread for everyone.                                                                                                                |
| The broadcast loop skips when nothing is attached        | No clients or no attached channels means no line is sent at all — which is why a completely unplugged rig produces silence rather than a stream of zeros.                                                                                                         |
| The recordings download endpoint ignores the downsampler | The chart endpoint thins deliberately; an export must not. Someone exporting "their data" gets the file that was written at save time, every sample.                                                                                                              |

## 9. Hardware realities

The physical rig does not behave like an ideal data source, and several decisions in `phidget_stream.py` exist only because of that.

- **Channels are opened non-blocking.** The backend starts, and runs, with no hardware present. Channels attach whenever boards appear. This is why the frontend waits 2 s after the socket opens before judging how many channels are present.
- **A detached channel is forced to 0.0**, not left at its last reading. A frozen last value is indistinguishable from a real constant force in a chart and in a recording; a hard zero at least reads as obviously wrong.
- **Partial attachment is normal and is warned about.** The banner names the sensor ("Only 9 of 12 sensor channels are attached — check: Right Hand"), and missing channels read 0.
- **Flask runs with `use_reloader=False`.** The reloader forks a second process that would also open the channels; single-open means both processes then fail to attach and the symptom looks like broken hardware.
- **Network-server discovery is deliberately off.** With the driver working, a channel silently attaching through a Phidget Network Server instead of USB would be nondeterministic and miserable to debug.

Hardware is never the ideal on paper. Load cells drift, amplifiers have offsets, a board that has been re-bolted reads differently. The calibration wizard and the live zero are not ceremony to be optimised away — they are the knobs that absorb the difference between the model and the wall.



## 10. Checking your work

There is no test framework, and that is a considered choice rather than neglect. What exists instead:

1. **The build gate.** `npm run build` runs `tsc` in strict mode (`noUnusedLocals`, `noUnusedParameters`) and then bundles. It catches most breakage. This is *the* gate — run it before every commit.
2. `wallDeclineSelfCheck()` in `wallGeometry.ts`. Three assertions covering the rotation model:  $\theta = 0$  is the identity, a pure vertical load resolves to all- $X$ , a pure normal push splits as  $(-\sin\theta, \cos\theta)$ . It runs automatically in the browser console in DEV. **Extend it when you touch the geometry** rather than adding a test runner.
3. `backend/test_recordings.py`. Plain asserts, run with `python test_recordings.py`. It covers the recording window/downsample arithmetic, whose edge cases (reversed bounds, out-of-range bounds) are reachable by dragging a range slider.
4. `backend/board_diagnostic.py`. Not part of the app: a forensic tool that answers "is this board's problem calibration, or is it mechanical?" from a recording of known-magnitude pulls. Its argument is that a calibration fault can only rescale each axis independently, so if no per-axis scaling reproduces the known applied magnitudes, recalibrating will never fix that board.
5. **The manual smoke path**, when hardware is attached: connect, confirm 12 of 12 channels, zero with the wall unloaded, hang a known weight and check the Newtons are sane, record, save, reload the recording from the Recordings tab.

Both automated gates were green on 2026-08-20, with the in-flight recordings-browser changes described in section 12 present in the tree.

If you add non-trivial logic — a parser, a loop, a piece of physics — leave one runnable check behind it, in the style already used here: an `assert`-based `demo()` next to the code, or three lines added to `wallDeclineSelfCheck()`. Do not introduce a test framework to hold six assertions.

## 11. Traps

| Thing                                                     | Why it bites                                                                                                                                                                                                |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>allSensorDataRef.current</code> is mutated in place | Handing it to React state means no re-render (same reference) and an array changing underneath <code>Chart.js</code> .                                                                                      |
| Mock mode bypasses <code>parseSerialLine</code>           | So <b>Zero Sensors</b> stays disabled with no hardware — the tare ring is never fed. Not a bug.                                                                                                             |
| Vite ignores <code>calibration_settings.json</code>       | The backend writes <code>src/config/calibration_settings.json</code> on every calibration save. Without the ignore in <code>vite.config.ts</code> , Vite full-page-reloads and drops the stream mid-wizard. |
| Flask runs <code>use_reloader=False</code>                | Two processes cannot both open single-open boards.                                                                                                                                                          |
| <code>calibration_profiles.json</code> is gitignored      | The profile library is local-only. A fresh clone starts with none, and nothing warns you.                                                                                                                   |
| The Vite proxy targets <code>127.0.0.1</code>             | Not <code>localhost</code> . IPv6 resolution order versus Flask's IPv4-only bind.                                                                                                                           |
| IndexedDB is at store version 2                           | The <code>v1</code> $\rightarrow$ <code>v2</code> upgrade <i>drops</i> the buffer, because the canonical frame changed from world to sensor. Stale data must never be reinterpreted.                        |
| Recording CSVs hold calibrated Newtons                    | Not raw ratios. A bad calibration at capture time cannot be undone later.                                                                                                                                   |



## 12. Open questions and rough edges

---

This is the honest part of the handover: things that are unresolved, unverified, or simply unfinished as of 2026-08-20.

### 12.1. The jump projection axes — verify before trusting absolute heights

`calculate_jump_height_with_angle` projects the summed hand and foot forces onto a global vertical axis like this:

```
foot_global_z = foot[:,2]*cos( $\theta$ ) + foot[:,1]*sin( $\theta$ )
```

Column 2 is  $Z$  (wall normal) and column 1 is  $Y$  (sideways). But the frontend sends sensor-frame triples  $[X, Y, Z]$ , and under this project's own convention the weight-bearing axes are  $X$  (in-plane,  $\approx W \cos \theta$ ) and  $Z$  (normal,  $\approx W \sin \theta$ ), while  $Y$  is lateral and physically independent of the wall tilt. A tilt projection in the  $X$ – $Z$  plane that consumes  $Y$  and ignores  $X$  does not obviously correspond to the geometry documented everywhere else.

The recordings support the convention rather than the code: in `peter_test_1_20260812_125458.csv` the large mean loads sit on  $X$  ( $-83, -135, -54$  N across the three live boards) with  $Z$  at  $+37$  to  $+49$  N and  $Y$  small, exactly as a hang on a  $16^\circ$  overhang should look.

This may still be intentional — the function's docstring is ambiguous about whether the height it returns is vertical or wall-perpendicular, and it applies a further  $\cos \theta$  at the end. Do not "fix" it from the armchair. **Measure a jump of known height and compare** before changing anything; then, whichever way it comes out, write the answer down in `measuring_pipeline.md`. Until then, treat absolute jump heights as unvalidated — which is also what the UI says, since the tab is labelled **Jump Test (BETA)**.

Two smaller things in the same function: the "first 1 second" comment sits above `total_force_z[:still_end * 3]`, which is three seconds; and `i_max_accel` is the argmax of foot *force*, not acceleration. Neither is necessarily wrong, but both will mislead the next reader.

### 12.2. The Right Foot board

The fourth GUI slot (S4, Right Foot, bottom right) reads flat zero through recent captures, and the commit history is blunt about it ("bottom right board is malicious"). `board_diagnostic.py` exists because this was being chased. Before blaming code for a missing channel, check the attach count in the warning banner and run the diagnostic against a known-weight recording.

### 12.3. Work in flight

The working tree on branch `minimalstic_code` carries an unfinished recordings browser: listing all recordings rather than the newest five (?limit=N now opt-in), rename, delete, verbatim CSV download, and a [from, to) row window on the data endpoint so zooming actually raises resolution instead of magnifying the same thinned points. The window arithmetic is covered by `test_recordings.py`; the UI (`RecordingsBrowserModal.tsx`) is newer than the rest. Finish or discard it deliberately — do not let it rot in the tree.

### 12.4. Smaller items

- The frontend bundles to a single  $\sim 820$  kB chunk. Nobody has needed to care yet; it is a lab tool on a local network, not a public site. `katex` is in there to render the wizard's formulas.
- `useCalibration.ts`'s header comment still names `captureAxis` and `commitAxis`; the real exports are `captureAverages` and `commitChannels`. Typical drift — fix it as you pass.
- `docs/` is gitignored, as noted in section 1.

## 13. House rules

---

Conventions this codebase actually follows. Match them and your changes will read like they belong.

- **Comments explain *why***, never what. A physical ceiling, a non-obvious threshold, a trap that has already been hit once. Migration history ("this used to be on the Pi") belongs in git, not in a file.
- **No line numbers in documentation.** They rot within a single commit. Reference the file and the function name.
- **Prefer configuration over code.** The board mapping, sample rate, and gain all live in `phidget_config.json` specifically so that the most common field fix requires no developer.
- **Keep the four Markdown docs in step with the code.** They are the contract the next person reads; a stale doc is worse than no doc.
- **Smallest change that works.** This codebase is roughly 9 500 lines including the backend, and it stays legible because features were repeatedly cut rather than added. Before building an abstraction, check whether the standard library, the platform, or an already-installed dependency covers it.

## 14. Your first hour

---

1. Start both servers and open the dashboard in demo mode. Press **Start Recording** and watch the charts fill.
2. Read `measuring_pipeline.md` end to end with the app open beside it. It is the one document that will save you the most time.
3. Run both self-checks: `npm run build`, and `python backend/test_recordings.py`. Open the browser console and confirm `WallDeclineSelfCheck()` reports three passes.
4. Open the Recordings tab and load a saved capture from `backend/saved_recordings/` — real data, real shapes, and a good feel for what the numbers should look like.
5. Read `serialParser.ts` in full. It is 84 lines and it is the heart of the system.

Written 2026-08-20, against branch `minimalstic_code`. When it goes stale, edit it — it is a source file like any other.